

Echelon UI Infrastructure

Co możemy zbudować przy pomocy frameworka @echelon-framework

dealer-fx-app · zespół

2026-05-05

Echelon — Infrastruktura UI

Dokument opisuje **wszystko**, co można zbudować przy pomocy frameworka @echelon-framework/* . Skupia się na praktycznych klockach UI, ich możliwościach konfiguracyjnych i miejscach styku z aplikacją (dealer-fx-app).

1. Filozofia i model

Echelon to **config-driven framework** dla aplikacji Angular 21+:

- **Strony** opisane jako PageConfig (deklaratywny JSON/TS) — bez ręcznego pisania komponentów
- **Widgety** wybierane z rejestru po type: string
- **Datasources** to abstrakcja — strona widzi tylko id; pod spodem może być HTTP, WebSocket, mock, computed lub pipeline
- **Designery** (wizualne edytory) i **Runtime** (renderer dashboardów) to dwa SDK z tej samej bazy
- **Eksport** PageConfig → JSON/TypeScript (bundle do zassania jako stała w aplikacji)

Wszystko jest **typowane** (TypeScript strict), **walidowane runtime** (Zod), **stylowane przez CSS variables** (multi-theme), **lokalizowane** (i18n keys w configu).

2. Model strony — PageConfig

Pakiet: @echelon-framework/core (src/config/index.ts)

PageConfig to “źródło prawdy” pojedynczego ekranu. Zawiera:

Sekcja	Opis	Co można
widgets: Record<id, WidgetConfig>	drzewo widgetów do wyrenderowania	layout 12-col grid lub free positioning
datasources: Record<id, DatasourceConfig>	źródła danych	HTTP, WS, mock, computed, pipeline
computed: Record<id, { expr, deps }>	derived values	wyrażenia z @echelon-framework/expression
eventHandlers: EventHandlerConfig[]	reakcje na eventy	on: 'name' → do: [actions]
lifecycle: { mount, unmount, beforeUnload, ... }	hooks cyklu strony	inicjalizacja DS, prefetch, cleanup
actions: Action[] per widget	akcje (przyciski, menu)	emit, setDatasource, clearDatasource, call(fn), navigate

Bindings w widget props używają wyrażeń: - `$ds.<id>` — wartość z datasource - `$ds.<id>[<index>].<field>` — indeksowanie tablic - `$computed.<id>` — wynik computed - `$session.<key>` — wartość z DI tokenów (np. user/role) - `$event.<key>` — payload zdarzenia w handlerze - `$local.<key>` — local component state

Przykład minimalnej strony:

```
export const clientsPage: PageConfig = {
  page: {
    id: 'clients',
    title: 'Klienci',
    datasources: {
      clients: { kind: 'transport', transport: 'http', endpoint:
        '/api/clients' },
    },
    widgets: {
      grid: {
        type: 'data-grid',
        bind: { rows: '$ds.clients' },
        options: { columns: ['name', 'email', 'phone'] },
      },
    },
  },
};
```

3. Widżety runtime — @echelon-framework/widgets-core

26 komponentów standalone (Angular). Każdy ma selector, @Input()/Output(), działa z OnPush change detection.

3.1. Strukturalne (shell, layout)

Selektor	Plik	Co robi
<ech-app-shell>	app-shell.component.ts	Główny szkielet aplikacji: header + sidebar + content
<ech-detail-shell>	detail-shell.component.ts	3-kolumnowy layout dla edycji: nawigator + canvas + inspektor
<ech-sidebar>	sidebar.component.ts	Boczny pasek nawigacji
<ech-context-sidebar>	context-sidebar.component.ts	Sidebar zależny od aktualnego selectiona
<ech-page-toolbar>	page-toolbar.component.ts	Toolbar z tytułem + akcjami strony
<ech-section-header>	section-header.component.ts	Nagłówek sekcji (heading + actions)
<ech-tab-strip>	tab-strip.component.ts	Pasek tabów (kontener + emit zmiany)
<ech-actions-bar>	actions-bar.component.ts	Pasek przycisków akcji (z grupowaniem)
<ech-modal>	modal.component.ts	Modal dialog (open/close, title, footer slot)
<ech-detail-drawer>	detail-drawer.component.ts	Boczne wysuwane okno detali

3.2. Dane (listy, tabele, drzewa)

Selektor	Co robi
<ech-data-grid>	Generic grid z kolumnami/sortowaniem/paginacją (lekka tabela)
<ech-data-table>	Bardziej rozbudowana tabela: zaznaczanie, akcje per-wiersz
<ech-editable-table>	Inline-edit cell (single field per row)
<ech-entity-list>	Lista encji z avatar/title/subtitle/actions
<ech-entity-header>	Nagłówek dla widoku detali encji (avatar, title, status)
<ech-pagination>	Sterowanie paginacją (page/total/size)
<ech-nav-tree>	Drzewo nawigacji (rozwijane)
<ech-menu-tree>	Drzewo menu z ikonami i akcjami
<ech-kv-list>	Lista par klucz-wartość (np. metadane)
<ech-bool-chips>	Chipsy boolean (status flags)
<ech-info-card>	Karta informacyjna (tytuł, body, akcja)
<ech-code-block>	Wycinek kodu z highlight

3.3. Formularze (Form Builder)

Pakiet: widgets-core/src/form-builder/

Selektor	Co robi
<ech-form-builder>	Runtime widget renderujący FormDefinition (z designer-core)
<ech-field-text>	Pole text (z prefix/suffix/maxLength/pattern)
<ech-field-email>	Pole email z natywną walidacją
<ech-field-password>	Pole password (toggle show/hide)
<ech-field-textarea>	Wieloliniowe pole z autosize
<ech-field-number>	Numeryczne (min/max/step/precision)
<ech-field-boolean>	Checkbox/toggle
<ech-validated-form>	Wrapper z walidacją na blur/submit (de- precated — używaj form-builder)
<ech-profile-form>	Pre-zbudowany formularz profilu (wzorzec)
<ech-filter-form>	Formularz filtrów (płaski model query params)
<ech-searchable-select>	Select z wyszukiwaniem (combobox)
<ech-form-ref>	Reference do innego formularza (composite/embed)
<ech-lookup-field>	Pole lookup (dropdown z DS, paginacja)

3.4. Adaptery Material/Bootstrap

widgets-core/src/material/ — opakowania mat-card, mat-form-field itd. dla
spójnego stylowania w obrębie Echelon themes (CSS variables).

4. Form Builder — szczegóły

Pakiet: @echelon-framework/designer-core (typy) + widgets-core/src/form-builder/ (runtime)

4.1. FormDefinition

```
interface FormDefinition {
  id: string;
  title?: string;
  description?: string;
  fields: FormFieldNode[];           // pola (drzewo via nested fields)
  layout?: FormLayout;              // 'single' | 'tabs' | 'wizard' |
    'sections'
  validators?: FormValidatorRef[];  // cross-field validators
```

```

submitLabel?: string;
showReset?: boolean;
showCancel?: boolean;
}

```

4.2. Typy pól (FormFieldNode)

Field-type-catalog (designer-widgets/src/form-field-type-catalog.ts):

- **Tekstowe:** text, email, password, url, tel, textarea
- **Numeryczne:** number, integer, decimal, currency, percentage
- **Boolean:** boolean, checkbox, toggle
- **Wybór:** select, multi-select, radio, chips
- **Daty:** date, datetime, time, date-range
- **Pliki:** file, image, multi-file
- **Złożone:** composite (zagnieżdżony obiekt), repeater (lista), lookup (DS-backed)
- **Specjalne:** markdown, code, color, slider, rating

Każde pole ma: - id, label, placeholder, description, hint - required, readonly, disabled, visible (boolean lub expr) - defaultValue - validators: ValidatorRef[] (per-field) - config (discriminated union per kind — FormBuilderTextFieldConfig, FormBuilderNumberFieldConfig, etc.)

4.3. Walidatory

Pakiet: @echelon-framework/designer-core (form-validators.ts)

Built-in: required, minLength, maxLength, min, max, pattern, email, url, equals, oneOf, customExpr.

Custom: zarejestrowane w DI przez provideEchelonFns({ validators: { myValidator: (val) => string|null } }).

Walidacja runtime: FormValidationResult z valid: boolean, errors: FieldValidationError[] (severity: error/warning/info).

4.4. Layouty

- single — wszystkie pola jeden pod drugim (12-col grid)
- tabs — taby grupujące pola
- wizard — kroki z prev/next + walidacja per krok
- sections — sekcje z nagłówkami (rozwijane)

4.5. Field Renderer Registry

Każdy typ pola ma rendererem zarejestrowanym w FieldRendererRegistry (form-builder-tokens.ts). Aplikacja może **zarejestrować własne renderery** dla custom typów pól:

```

provideEchelonFormBuilder({
  customRenderers: { 'my-custom-field': MyCustomFieldComponent },

```

```
});
```

5. Datasources — abstrakcja

Pakiet: `@echelon-framework/designer-core` (`datasource-implementations.ts`, `datasource-resolver.ts`)

5.1. Model DatasourceAbstraction

DS to **abstrakcja** — strona pyta o `$ds.clients`, framework rozwiązuje przez aktywną `implementation`. Możliwość trzymania wielu implementacji per DS (draft/staging/production):

```
interface DatasourceAbstraction {
  id: string;
  schema: { properties: Record<string, PropertyDef> };
  implementations: DatasourceImplementation[]; // wiele wariantów
  activeImplementationId: string; // aktualnie
  używany
}
```

5.2. Rodzaje implementacji (ImplementationConfig.kind)

Kind	Co robi	Use case
transport	HTTP request	REST API (endpoint, method, params)
local	dane inline (statyczne)	seed data, lookup tables
computed	wynik funkcji TS	agregacje, derived state
pipeline	łańcuch operacji	filter → sort → paginate (ETL)
mock	sztuczne dane (z behavior)	development, demo (delay, errorRate)
stream	WebSocket subscribe	real-time (FX rates, notifications)

5.3. Query operations (mock-server + transport)

`mock-server` (`@echelon-framework/mock-server`) i `transport-http` obsługują query params:

- `?q=text` — full-text search po wszystkich polach
- `?<field>=value` — per-field filter (contains, AND między polami)
- `?_page=N&_pageSize=K` — paginacja
- `?_sort=field&_order=asc|desc` — sortowanie

Czyli grid + filter-form + pagination “po prostu działają” przez query string.

5.4. Pipeline DS

Łańcuch transformacji nad innym DS lub source:

```
{
  kind: 'pipeline',
  source: { kind: 'datasource', id: 'allClients' },
  steps: [
    { kind: 'filter', expr: 'role === "premium"' },
    { kind: 'sort', by: 'name', order: 'asc' },
    { kind: 'paginate', page: 1, size: 20 },
  ]
}
```

6. Transport — HTTP / WS / Mock

6.1. @echelon-framework/transport-http

Wraps fetch z retry, AbortSignal, base URL, headers, JSON parsing. Token: TRANSPORT (InjectionToken).

6.2. @echelon-framework/transport-ws

WebSocket adapter — subscribe/unsubscribe na kanały, auto-reconnect, message buffer.

6.3. @echelon-framework/transport-mock

Adapter Mock w pamięci aplikacji — bez real network. push(channel, data) emituje do subskrybentów. Dla testów.

6.4. @echelon-framework/mock-server (Node CLI)

Standalone Node serwer — HTTP + WebSocket. Konfiguracja mock-server.json:

```
{
  "port": 3001,
  "responses": {
    "clients": [{ "id": "c1", "name": "Acme" }, ...]
  },
  "streams": {
    "rates.usdpln": { "simulate": { "kind": "fx-random-walk",
      "intervalMs": 1000, "mid": 4.05, "vol": 0.0003 } }
  }
}
```

- GET /health → {status: ok}
- GET /api/<resource> → fixture (z query ops)
- POST /api/<resource> → echo + journal

- `ws://host:port/ws` → kanały via subscribe

Symulatory streamów: **fx-random-walk**, **sine**, **random-int** (z parametrami: bounds, jumpProbability, meanReversionTarget, precision, amplitude, periodMs).

7. Charts

7.1. @echelon-framework/charts-core

Adapter-pattern. Definiuje `ChartConfig` i `ChartAdapter` (DI token `CHART_ADAPTER`).

7.2. @echelon-framework/charts-echarts

Konkretna implementacja na ECharts.

Rodzaje wykresów (`ChartKind`): line, bar, area, pie, gauge, scatter, candlestick, heatmap.

ChartSeries: dane, type, style, axis assignment. **ChartAxisConfig:** x/y axis (kategoryczna, czasowa, numeryczna, log). **ChartAnnotation:** linia, region, marker (dla highlighting). **ChartTheme:** paleta, fonts, grid colors (zsynchronizowane z themes Echelon).

Strona wstawia `<ech-chart [config]="$computed.priceChartConfig" />` (przez widget chart).

8. Designery (wizualne edytory)

Pakiet: `@echelon-framework/designer-widgets`

Każdy designer to **standalone Angular component**. Aplikacja decyduje czy je włączać (przez `designerPages()`, `designerMenuItems()`, `designerWidgets()`).

Designer	Co edytuje
<code><fx-application-designer></code>	Unified canvas — jeden ekran, lewy panel = navigator (modele, procesy, DS, strony, pipeline, formularze), centrum = kontekstowy edytor, góra = BPMN flow procesu
<code><fx-model-designer></code>	Modele danych — pola (id, label, type, required, PK, serverManaged, enumValues), relacje 1:1/1:N, schema preview
<code><fx-form-designer></code>	Formularze — Form Builder z drag&drop, field inspector, layout (single/tabs/wizard)

Designer	Co edytuje
<code><fx-datasource-designer></code>	Datasources — wszystkie 6 kindów, test panel z response viewer, multi-implementation switching
<code><fx-process-designer></code>	Procesy biznesowe — BPMN canvas (start, end, user-task, service-task, gateway), step editor (model+form+ds+pre-view+actions), sub-procesy
<code><fx-pipeline-designer></code>	Pipeline DS — wizualny edytor łańcucha transformacji
<code><fx-menu-editor></code>	Menu aplikacji — drzewo z ikonami, route bindings, guard configuration
<code><fx-theme-manager></code>	Themes — wybór built-in + override CSS variables, live preview
<code><fx-translation-manager></code>	i18n — klucze tłumaczeń, języki, eksport/import
<code><fx-export-panel></code>	Eksport bundle JSON / TypeScript
<code><fx-page-wizard></code>	Wizard tworzenia nowej strony (z szablonów)
<code><fx-designer-shell></code>	Shell zawierający wszystkie powyższe

8.1. Process Designer — BPMN

@echelon-framework/designer-core (process-types.ts, process-generator.ts):

Step kinds: start, end, user-task, service-task, gateway.

Generator: z procesu generuje **wszystkie artefakty** (modele, formularze, DS, strony, route, mock data) — single source of truth.

Sub-procesy: drill-in / drill-out, zachowanie mock-data on step.

9. Themes

Pakiet: @echelon-framework/designer-widgets (css-themes.ts)

7 built-in: - dark-default, dark-midnight - light-clean, light-corporate - contrast-high (ally) - bnp-light, bnp-dark (branding)

Każdy theme = zestaw CSS variables (--ech-bg, --ech-fg, --ech-accent, --ech-border, --ech-panel, --ech-muted, --ech-danger, --ech-warning, --ech-info).

API: applyTheme(theme), getCurrentThemeId(), loadSavedTheme() (localStorage).

Custom theme = nowy obiekt CssTheme zarejestrowany w provideEchelon({ themes: [...customThemes] }).

10. i18n — translacje

Pakiet: `@echelon-framework/designer-core` (`draft-translation-store.ts`)

Klucze tłumaczeń trzymane w `DraftTranslationStore` (`localStorage` w `design-time`, `bundle` w produkcji). Format JSON:

```
{
  "pl": { "form.save": "Zapisz", "form.cancel": "Anuluj" },
  "en": { "form.save": "Save", "form.cancel": "Cancel" }
}
```

W configu strony używa się kluczy `t:form.save` (resolver w runtime).

11. Eksport bundle

Pakiet: `@echelon-framework/designer-widgets` (`export-bundle.ts`)

Eksport wszystkich draftów (modele, DS, formularze, strony, procesy, pipeline'y, theme, i18n) jako:

- **JSON** — single file, deklaracyjny format do zassania w runtime
- **TypeScript** — typowane stałe gotowe do import w kodzie aplikacji

```
const bundle = exportBundle();
exportBundleAsJson(bundle);           // pobiera plik .json
exportBundleAsTypeScript(bundle);     // pobiera plik .ts
```

Aplikacja używa bundle jako **read-only source** (po `git commit` w repo) lub **hot-load** (zassany w runtime przez `fetch`).

12. Runtime host

Pakiet: `@echelon-framework/runtime`

12.1. <ech-dashboard-renderer>

Główny komponent — bierze `PageConfig` i renderuje. Otwartość: aplikacja providuje rejestry przez DI (widgets, charts, transports).

12.2. Komponenty wewnętrzne

- `port-resolver` — rozwiązuje `PortSource` (literal, datasource, session, transform) na `Observable`
- `binding-engine` — interpretuje `bind: { prop: '$ds.foo' } → real values`
- `dashboard-registry` — DI token z mapą `type → ComponentClass`
- `data-bus` — pub/sub dla DS values

- `event-bus` — pub/sub dla event handlers

12.3. DI tokens

- `DATA_BUS`, `EVENT_BUS`, `ERROR_BUS`, `LOGGER`, `CLOCK`
- `WIDGET_REGISTRY`, `PAGE_REGISTRY`, `TRANSPORT`
- `COMPUTED_FUNCTIONS`, `VALIDATORS`, `FORMATTERS`, `PREDICATES`

Aplikacja providuje przez `provideEchelon({...})`.

12.4. Lifecycle phases (per-page)

`mount`, `beforeUnload`, `unmount`, plus events `emit`/`listen` w trakcie życia.

13. Functions / plugins

13.1. @echelon-framework/functions-core

Aplikacja deklaruje:

```
provideEchelon({
  validators: { myValidator: (v) => /* string | null */ },
  formatters: { currency: (v) => `${v} PLN` },
  predicates: { isPremium: (user) => user.tier === 'premium' },
});
```

Te funkcje są dostępne w configu przez nazwę: `bind: { value: '$call.currency($ds.price)' }`.

13.2. @echelon-framework/eslint-plugin

Custom ESLint rules dla typowania bindings, sprawdzania referencji DS, walidacji `PageConfig` w design-time.

14. Embed

Pakiet: `@echelon-framework/embed`

Echelon page jako **iframe-embeddable** widget. Host-bridge (`postMessage`) — strona zewnętrzna może: - Załadować dashboard (`{type: 'load', config: PageConfig}`) - Emitować eventy (`{type: 'emit', event, payload}`) - Subskrybować eventy z dashboardu (`{type: 'subscribe', event}`)

Use case: osadzenie dashboardu w innym serwisie (CRM, ERP).

15. CLI + schematics

15.1. @echelon-framework/cli

Standalone Node CLI: - echelon validate <bundle.json> — sprawdza schemat (Zod) - echelon migrate <bundle.json> — migracja starszych wersji bundle - echelon generate <process> — z procesu generuje strony/DS/forms

15.2. @echelon-framework/schematics

Angular schematics: - ng add @echelon-framework — bootstrap aplikacji z provide-Echelon - ng generate @echelon-framework:page <name> — scaffold nowego PageConfig

16. Expression engine

Pakiet: @echelon-framework/expression

Mini-evaluator JS-podobny (zero-dep, < 200 linii). Obsługuje:

- Literałów: 42, "text", true, null
- Identyfikatory: foo, foo.bar, arr[0]
- Arytmetyczne: +, -, *, /, %
- Porównania: ==, !=, <, <=, >, >=
- Logiczne: &&, ||, !, ??
- Wywołania funkcji: myFn(a, b) (z provideEchelon({ functions }))
- Indeksowanie / property access
- Specjalne prefiksy: \$ds., \$computed., \$session., \$event., \$local.

Bezpieczeństwo: **brak eval**, brak new, brak globali, sandbox z deadline + timeoutMs.

17. Core utilities

Pakiet: @echelon-framework/core

Moduł	Co robi
compliance	audit log, GDPR helpers
storage	abstract over localStorage/sessionStorage/IndexedDB
event-bus	typed pub/sub
data-bus	reactive DS values
feature-flags	runtime-switchable flagi
plugins	rozszerzenia frameworka
workers	Web Worker bridges (heavy compute off-main-thread)

Moduł	Co robi
transport	base abstrakcja transport-* adaptery
widget	bazowe interfejsy WidgetConfig, WidgetManifest

18. Co możemy zbudować w dealer-fx-app

Konkretne use case'y z aplikacji dealer:

18.1. Lista klientów + edycja

- data-grid z bindem do `$ds.clients` (HTTP)
- searchable-select do wyboru klienta
- Klik → `<ech-detail-drawer>` z `<ech-form-builder>` (FormDefinition: imię, email, telefon, adres composite)

18.2. Spot FX trade

- chart (line + candlestick) z `$ds.rates.usdpln` (WS stream → transport-ws)
- `<ech-form-builder>` z polami: para walutowa (lookup z DS), kwota (number z step=0.01), strona (select buy/sell), klient (lookup)
- `<ech-actions-bar>` z przyciskami "Quote" → action call → DS pricing

18.3. Dashboard menedżera

- `<ech-app-shell>` z menu (built przez Menu Editor)
- Strony zbudowane z: `<ech-info-card>` (KPI), `<ech-chart>` (PnL graph), `<ech-data-table>` (top N transakcji)
- Multi-theme switcher w toolbar (dark/light/contrast/BNP)

18.4. Process — onboarding klienta

- BPMN flow: Start → Dane podstawowe → Verification (gateway) → Dokumenty → KYC service-task → End
- Każdy krok generuje stronę z formularzem (wygenerowane przez Process Designer)
- Mock data per krok dla preview

18.5. Eksport / hot-config

- Designer wyeksportuje bundle (TS/JSON) → commit do `dealer-fx-app/src/app/config/`
- Aplikacja zassie bundle → renderowanie bez deployu nowego kodu
- Lub: hot-load z backend (`fetch('/api/dashboard-config')`) → `<ech-dashboard-renderer>`

18.6. Embed w external system

- Wystawienie strony jako iframe widget (np. dla CRM)
- Host-bridge dla bidirectional communication (selection sync, action triggering)

19. Co NIE jest w scope (boundaries)

- **Backend / API server** — Echelon to klient + design-time, backend to oddzielny system (WebApi, Spring, Node) który aplikacja konsumuje
- **Database** — DS są abstrakcją; faktyczna baza i ORM to backend
- **Authentication / SSO** — token zostaje przekazany przez aplikację do transport-http headers, ale logika auth (OIDC, OAuth) to backend
- **Native mobile** — Angular only (PWA ✓, React Native ✗)
- **3D / WebGL** — charts są 2D (ECharts); 3D wymaga osobnej integracji

20. Wersje i kompatybilność

Pakiet	Wersja latest	RC next
@echelon-framework/core	0.6.0	0.7.0-rc.1
Wszystkie 21 pakietów	0.6.0	0.7.0-rc.1

Angular: 21+ (peer dep). Standalone components, signals, OnPush by default. **Node:** ≥24 dla buildu, runtime browser-only. **TypeScript:** 5.6+ strict.

dealer-fx-app aktualnie: ^0.6.0 (stable).

21. Strategia wymiany starego GUI przez embed

Echelon embed (rozdz. 14) umożliwia **stopniową migrację** legacy aplikacji bez big-bang rewrite. Zamiast przepisywać cały front naraz, zastępujemy go **kawałkami** — Echelon page-bundle osadzony w iframe wewnątrz starego shellu, z bidirectional bridge przez postMessage.

To wzorzec **strangler fig** — stara aplikacja “obraca” nowymi fragmentami, kurczy się, aż w końcu znika.

21.1. Założenia wstępne

- Stara aplikacja **musi pozwolić na osadzenie iframe** (CSP, X-Frame-Options) w obrębie własnej domeny lub jako allowed origin
- Komunikacja host ↔ embed jest **wyłącznie przez postMessage** — żaden bezpośredni dostęp do DOM/JS po obu stronach
- **Sesja użytkownika** (token, role, locale, theme) jest źródłem prawdy w starym hoście; embed tylko ją konsumuje, nie zarządza

- **Routing** pozostaje w gestii starego shellu na poziomie nawigacji globalnej; embed obsługuje routing wewnątrz osadzonego widoku

21.2. Cztery fazy migracji

Faza 1 — Przygotowanie infrastruktury (1–2 sprinty)

- Wystawienie po stronie hosta endpointa do dostarczania bundle’i (GET /api/dashboard/<id> → PageConfig)
- Implementacja host-bridge w starej aplikacji (warstwa komunikacji z iframe: pre-load auth, intercept events, propagacja zmian theme/locale)
- Zdefiniowanie **kontraktu komunikatów** (TypeScript types) wspólnego dla obu stron
- Wybór 1 niskoryzykowny ekran (read-only widget, np. dashboard, lista) jako proof of concept

Faza 2 — Pierwszy embed end-to-end (1 sprint)

- Wybrany ekran zbudowany w designerze Echelon → eksport bundle → publikacja
- Stara aplikacja zastępuje swój komponent iframe’em z <ech-dashboard-renderer> w środku
- Walidacja: parity functional (ten sam UX), parity stylistyczna (theme propagation), brak regresji w hoście
- Feature flag w hoście: embed:dashboard — natychmiastowy rollback do legacy

Faza 3 — Skalowanie (n sprintów, równolegle z developmentem)

- Tabela migracyjna: każdy ekran legacy → status (legacy / in-progress / embedded / removed)
- Granica priorytetów: read-only przed write, prosty CRUD przed flow procesowymi, ekrany z małym ruchem przed core flow
- **Każdy embed jest niezależnym deployem** bundle’a — bez konieczności redeployu hosta
- Stara aplikacja stopniowo traci kod: usuwany komponent legacy gdy jego ekran jest 100% w embed dla wszystkich użytkowników

Faza 4 — Inwersja (cel końcowy)

- Gdy wszystkie istotne ekrany są w Echelon, **stara aplikacja staje się shellem** — kontener nawigacyjny + auth + global state
- Następnie shell zostaje przepisany na natywny Echelon <ech-app-shell> lub minimalna wrapperka
- Stara aplikacja zostaje wyłączona

21.3. Co przechodzi przez bridge (kontrakt)

Komunikacja host ↔ embed jest **deklaratywna i typowana**. Cztery klasy komunikatów:

Kierunek	Typ	Cel
Host → Embed	init	Przekazanie configu + sesji + theme + locale przy starcie
Host → Embed	update	Aktualizacja sesji/theme/locale w trak-

Kierunek	Typ	Cel
Host → Embed	command	cie życia Polecenie wykonania akcji (refresh, focus, save)
Embed → Host	event	Zdarzenie domenowe (selection, navigation request, error)

Każda klasa ma zdefiniowany schemat (Zod). **Nieznane komunikaty są ignorowane** — wersjonowanie kontraktu jest przyrostowe, host i embed mogą być w różnych wersjach bez krachu.

21.4. Granice modułów

Dobry kandydat na embed: - Ekran z **wyraźną granicą domenową** (jeden agregat, jeden flow) - Konsumuje dane z **dobrze zdefiniowanego API** (REST/GraphQL/WS) - Komunikuje się ze starą aplikacją **przez ograniczoną liczbę zdarzeń** (selection, save complete, request close) - Może być wyświetlony w **prostokątnym kontenerze** (nie polega na overlay/popout poza iframe'em)

Zły kandydat: - Ekran z **gęstym sprzężeniem** ze stanem globalnym hosta (wspólny store, wspólne dialogi, wspólny event bus) - Wymaga **drag&drop poza granice iframe** (cross-frame drag jest skomplikowane) - Otwiera **modale na poziomie całej aplikacji** (powinien delegować to do hosta przez event) - **Custom keyboard shortcuts globalne** (kolizje host/embed)

Granice modułu wyznacza się **przed** rozpoczęciem migracji ekranu — jeżeli granica nie istnieje, najpierw refactor legacy.

21.5. Spójność wizualna

- **Theme** propagowany z hosta przez postMessage na starcie + przy zmianie. Echelon ma 7 wbudowanych motywów + override CSS variables — host przekazuje swoje wartości CSS variables, embed je stosuje
- **Typografia/odstęp** zharmonizowane przez wspólny `:root { ... }` token set — embed publikowany razem z odniesieniem do wersjonowanego “design tokens contract”
- **Z-index / layering** — embed pracuje wewnątrz swojej granicy iframe; modale globalne deleguje do hosta
- **Responsive** — embed jest “bezgraniczny” w sensie szerokości; host kontroluje wymiar przez CSS na iframe

21.6. Sesja, autoryzacja, lokalizacja

- **Token autoryzacyjny** przekazywany przez postMessage init — embed przepuszcza go do transport-http headers; nie trzymany w localStorage embed (single source of truth = host)
- **Refresh token** zarządzany wyłącznie po stronie hosta — embed reaguje na update.session gdy token się odświeża
- **Locale i18n** propagowany z hosta — embed używa DraftTranslationStore skonfigurowanego pod aktywny język

- **Logout** to command: `logout z hosta` — embed unmount + cleanup

21.7. Routing i nawigacja

- **Routing globalny** zostaje w starej aplikacji do końca (lub do fazy 4)
- Embed używa swojego routera **wewnątrz iframe** dla nawigacji w obrębie osadzonego widoku
- Embed prosi host o nawigację globalną przez `event:navigate-request` — host decyduje, czy ją wykonać (np. zmienić ekran legacy lub załadować inny embed)
- **Deep linking**: URL hosta zawiera identyfikator aktualnego embedu + jego wewnętrzny stan — embed deserializuje stan z `init.params`

21.8. Dane — DS i transport

- **Backend nie zmienia się** w trakcie migracji — embed konsumuje te same API co stara aplikacja
- **Datasource w embed** wskazuje na ten sam endpoint co legacy; abstrakcja DS pozwala później przełączyć na inny backend bez zmian w embed
- **Stream/WebSocket** — jeden socket per host (pojedyncze połączenie), embed dostaje selektywne wiadomości przez bridge (host filtruje i forwarduje)

21.9. Risk control

- **Feature flag per-embed** — `embed:<screen-id>` w hoście; gdy off → legacy fallback
- **Wersjonowanie bundle'i** — stara wersja bundle'a zostaje dostępna do natychmiastowego rollbacku (deploy nowego bundle nie kasuje poprzedniego)
- **Smoke test post-deploy** — automatyczna weryfikacja kluczowych eventów embedu w produkcji
- **Telemetria po obu stronach** — host loguje błędy z embedu (forwarded przez `event:error`), embed loguje czas inicjalizacji i błędy walidacji
- **Stopniowy rollout** — A/B test, canary % użytkowników, dopiero potem 100%

21.10. Wskaźniki sukcesu (per-embed)

- Brak regresji funkcjonalnej (E2E coverage tych samych przypadków co legacy)
- Czas startowego rendera $\leq$ poprzedni
- Krytyczne ścieżki (save, submit) działają end-to-end z bridge
- Error rate (errors/session) $\leq$ poprzedni
- Telemetria UX (czas wykonania zadania, abandonment) w granicach normy

21.11. Kiedy NIE iść w embed

- Aplikacja jest tak mała, że **przepisanie z gruntu zajmie mniej** niż infrastruktura embed (granica zwykle ~5-10 ekranów średniej złożoności)
- Backend wymaga jednoczesnej zmiany — wtedy migracja UI bez backendu daje pozytywny efekt
- Aplikacja będzie **wycofywana w < 12 miesięcy** — koszt embed-infra się nie zwróci
- **Brak dostępu do hosta** (zewnętrzny vendor) — wtedy drop-in replacement, nie migracja krokowa

22. Roadmap (znane TODO)

- **0.7.0:** extract runtime-tokens package — wycięcie cyklu designer-core ↔ runtime
 - **0.7.0:** pełen strict-mode lint (aktualnie disable directives jako TODO)
 - **0.7.0:** search filters w designerach, mock fixes, sub-processes UX
 - **Backlog:** React adapter (port widgets-core), CMS integration, A/B testing widgets
-

Kontakt

Repo: [git@gitlab.com:echelon-framework/echelon.git](https://gitlab.com:echelon-framework/echelon.git) TC:
<http://192.168.50.112:8111> (Verify, PublishRC, PublishGA) npm:
<https://www.npmjs.com/org/echelon-framework>

— koniec dokumentu